

Sequence Diagram

Understand how sequence diagrams represent interactions between actors and objects.

A **sequence diagram** is a form of communication diagram that illustrates how different actors and objects interact with each other or between themselves. The diagram represents these interactions as an exchange of **messages** between **various entities** and the type of exchange. Sequence diagrams also demonstrate the **sequence of events** that occur in a **specific use case** and the logic behind different **operations and functions**.

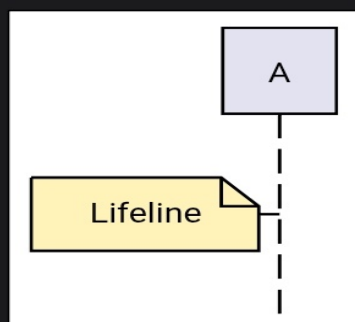
Elements of a sequence diagram

Various elements make up a sequence diagram. Let's discuss some of the essential elements of this diagram that appear most often.

Lifeline

In sequence diagrams, we write all entities horizontally. Each **entity** has a **lifeline** that represents its existence, i.e., when the entity is activated or deactivated. We illustrate this using a dotted line below the entity. We represent different objects, actors, entities, or boundaries in a system using lifelines, and they never overlap with each other. The illustration below shows how to visualize a lifeline in UML. The horizontal boxes are used to denote the objects involved

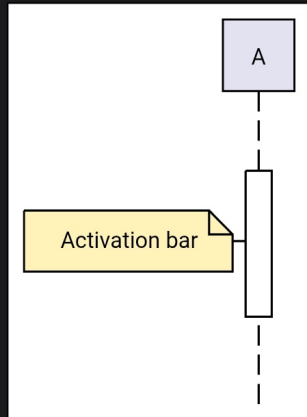
✦ Ask



Lifeline in a sequence diagram

Activation bars

Activation bars indicate the **active period of an object**, that is, the time when an **object sends or receives messages**. We draw **these using simple vertical boxes** on the **lifeline**. Here's an example of an activation bar:

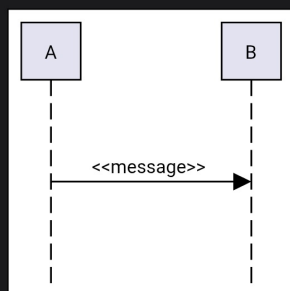


Activation bar in a sequence diagram

Messages

In sequence diagrams, a message is an interaction between two objects. It can be in the form of **sending and receiving messages**, **invoking an operation**, or **creating a new object or entity**. Messages are drawn horizontally in any direction: left to right, right to left, or back to themselves. Different kinds of messages are represented using different arrows. Let's look at the types of messages:

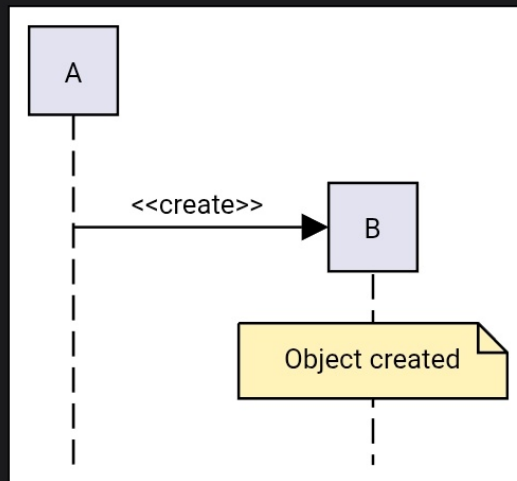
1. A **synchronous message** is a **type of message** where the **sender has to wait for the receiver to return a response** before it can perform another operation. We can draw synchronous messages using a solid line with a filled arrowhead.



A synchronous message

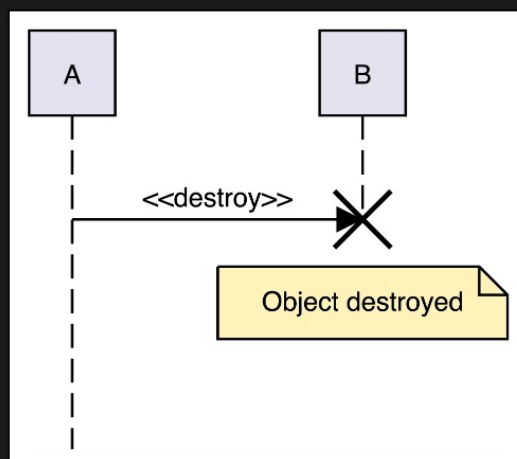
Diagram representation of synchronous message
" —————> " [closed arrow]

4. A **create message** indicates that a new object is created during an interaction. New objects can be created as a result of some message or operation. We can draw it like this:



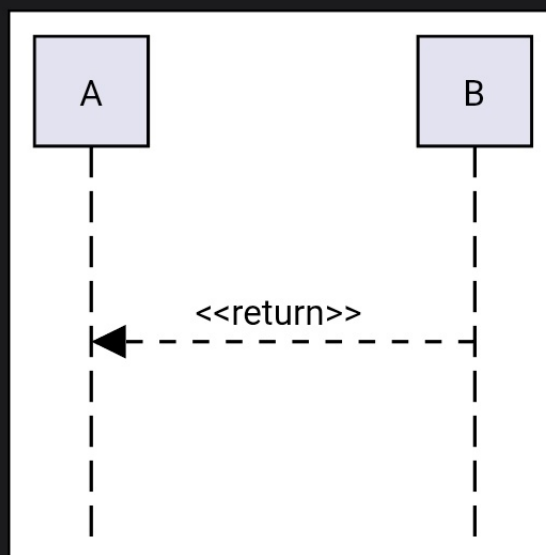
A create message

5. A **destroy message** indicates that an object is destroyed during a sequence of events. Objects can be destroyed as a result of some message or operation, and their lifeline ends. We can draw it like this:



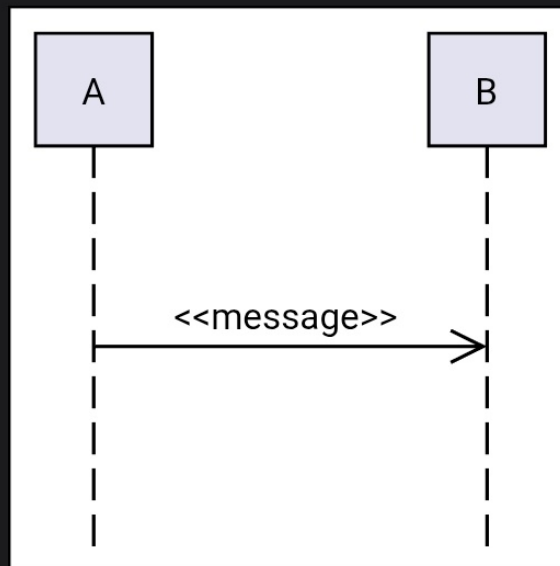
A destroy message

3. A **synchronous return** is a type of message generated in response to a synchronous call. A synchronous message has to be paired with a synchronous return message. It means that the receiver has processed the message and sent a response. We can draw synchronous return messages using a dotted line with a filled arrowhead.



A synchronous return message

2. An **asynchronous message** is a **type of message** where the sender **does not have to wait** for a response from the receiver. The **sender can continue sending messages to other objects**. We can draw the asynchronous messages using a solid line with an open arrowhead.



An asynchronous message

✦ Ask

Diagram representation of Asynchronous message
“ ———> ” [open arrow]

Identify the actors and objects

Now, we need to list down all the actors and objects that will be involved in the entire sequence. These would be the following:

- Customer
- ATM
- Transaction
- Account
- Cash dispenser

Identify the order of actions

We have identified the entities involved in the cash withdrawal sequence, but we don't know how these entities will interact with each other. It is important that we note down the action and the order in which they will occur. Here's how we can define the interaction in steps:

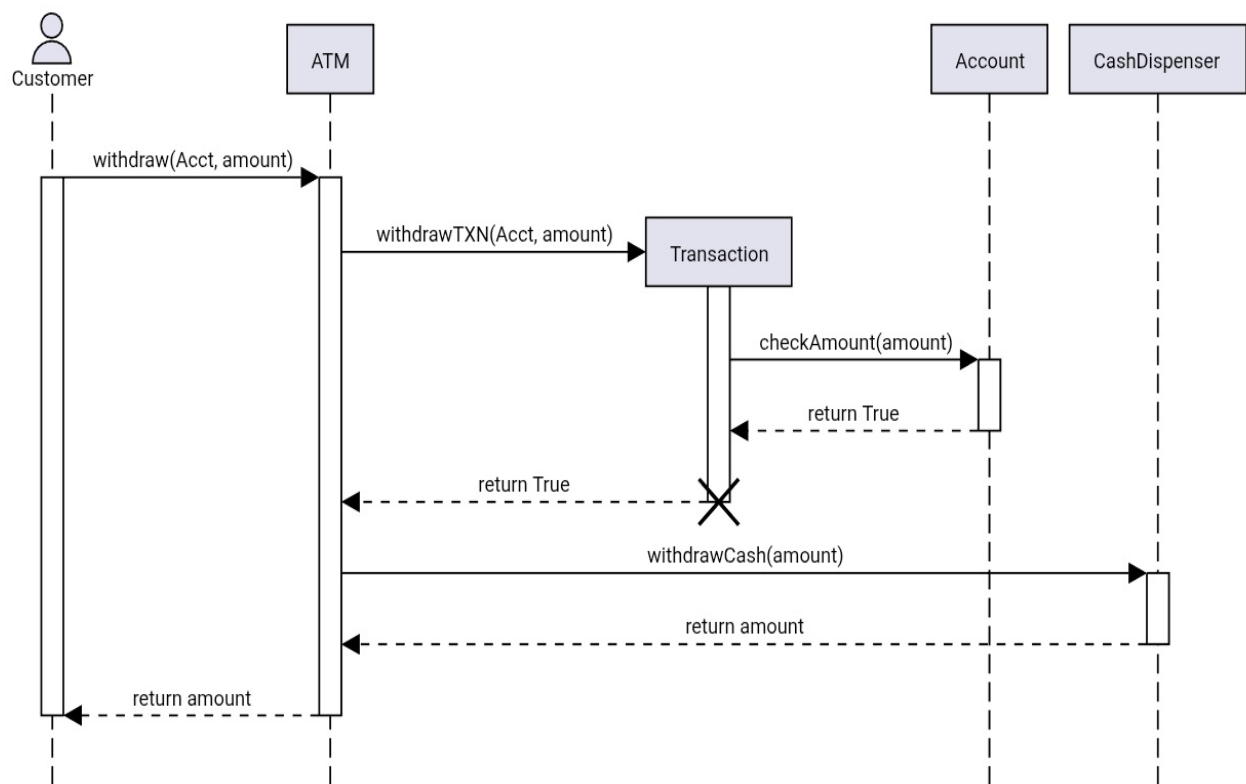
1. The customer requests a cash withdrawal from the ATM with their account information and the amount.
2. The ATM initiates a cash withdrawal transaction against the account and the given amount.
3. The transaction amount is verified for the given account.
4. In case the amount entered is valid, the account verifies the transaction.
5. The ATM signals the cash dispenser to release the required cash amount.
6. The cash dispenser confirms the release of cash.
7. The ATM prompts the user to collect the cash.

Create the diagram

We have identified all the involved objects and entities in the interaction. We have also listed down the order in which the objects will perform these actions. To finally create the sequence diagram, we need to relate these steps with the type of messages that the objects will exchange with each other. Here's a possible diagram that we can create for the cash withdrawal scenario:

✦⁺ Ask

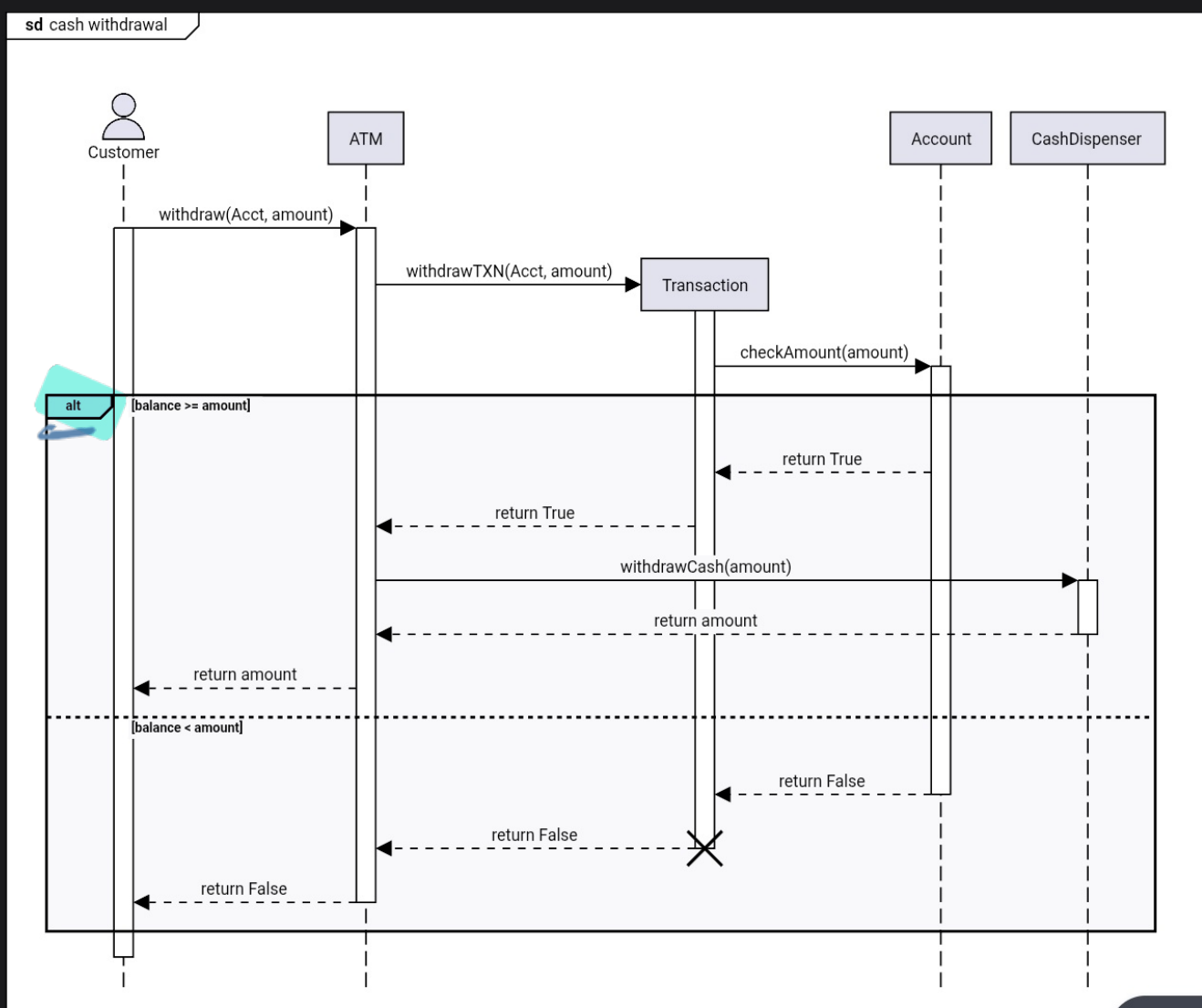
sd cash withdrawal



The sequence diagram for cash withdrawal

- **Alternative (alt):** This operator models the `if-else` condition. It divides the fragment into parts, and either of the parts can take place based on the guard condition.
- **Option (opt):** This operator models the `if` condition. The fragment will only execute if the guard condition is met. Otherwise, the entire fragment is skipped, and the interaction continues.
- **Loop (loop):** This operator represents a repetitive sequence. The fragment will keep repeating until the guard condition is met.
- **Reference (ref):** This operator assists in managing larger diagrams. It allows us to reuse parts of another sequence diagram by providing a reference to it. 

Let's expand the sequence diagram we created to see how we can build on it further using a sequence fragment.



The sequence diagram for cash withdrawal with a sequence fragment

Activity Diagram

Learn how activity diagrams represent interactions between actors and objects.

An **activity diagram** is a communication diagram that is used to show the dynamic aspects performed by a **system**. This diagram is used to **represent a series of actions**, similar to how they may appear in a flowchart.

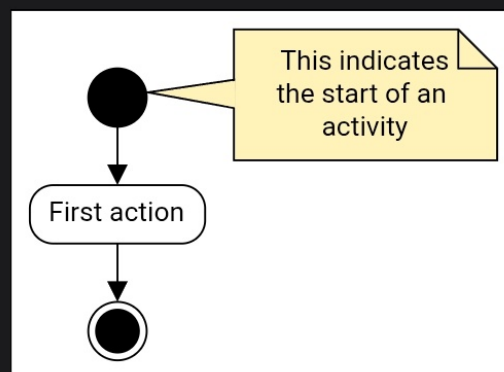
Purpose of activity diagram

Much like the previously discussed UML diagrams, activity diagrams are also used to show the sequence through which events occur in the system. **Activity diagrams differ from other UML diagrams** in that they specifically **capture the message flow for each activity on to the next**. These may look like flowcharts in appearance, but in execution, they can be much more. They can show various **flows, such as single, parallel, concurrent, and branched**.

Components of an activity diagram

Various components make up an activity diagram. Let's discuss some of the essential features of this diagram that appear most often.

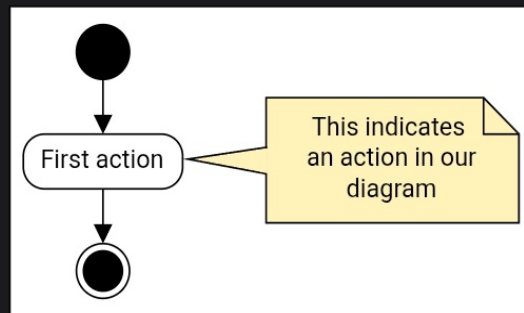
Initial: This represents the start of the workflow of the activity diagram. They can be visualized as the node in a tree structure.



Initial node in an activity diagram

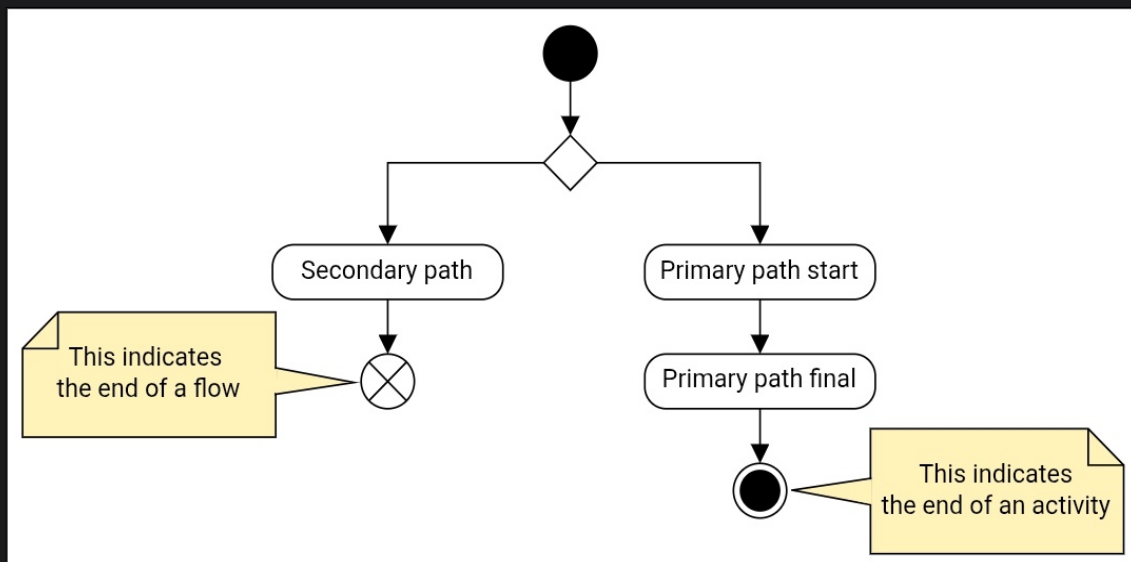
✦ Ask

Action: These are the main building blocks of an activity diagram and are used to show the activities that a modeled process is made of.



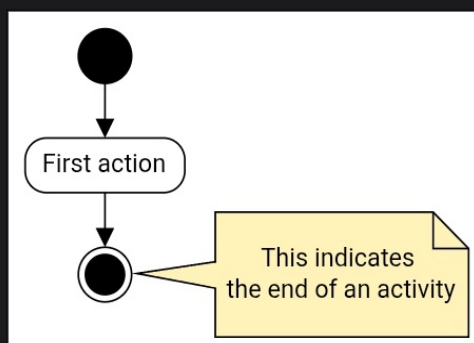
Action in an activity diagram

Flow final: This represents the end of a single path in the activity diagram. They can be visualized as a leaf in a tree structure.



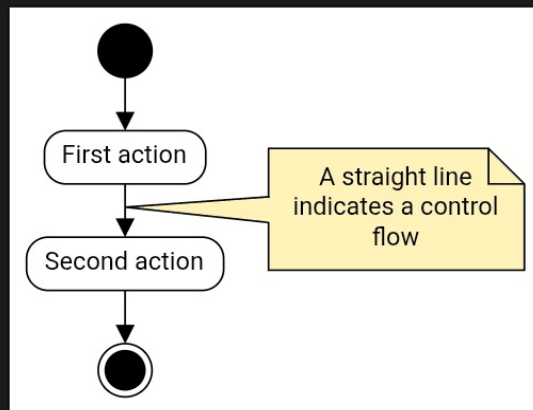
Flow final in an activity diagram

Activity final: This represents the end of all the activities in the activity diagram.



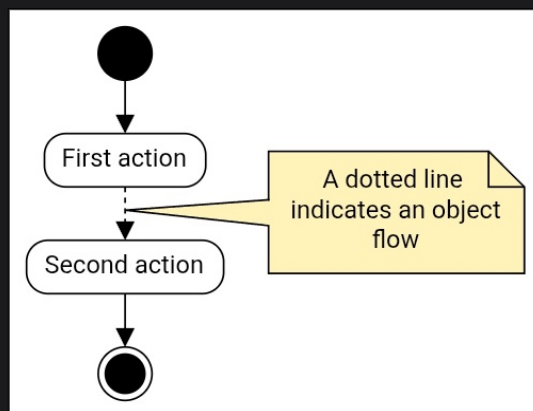
Activity final an activity diagram

Control flow: This shows the **directional flow of the diagram**. This exists as a **connector between one action and another**.



Control flow in an activity diagram

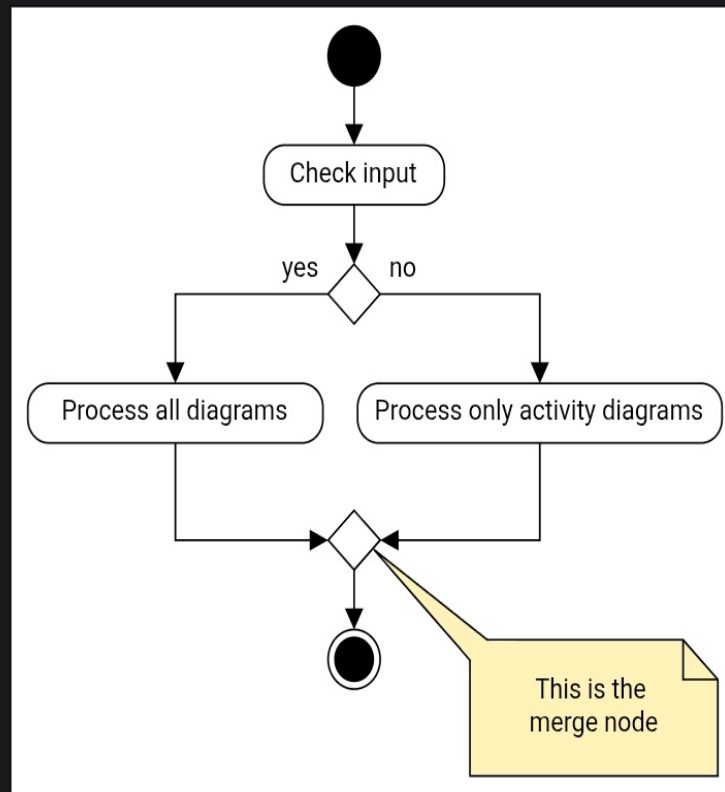
Object flow: This shows the **path of the objects** as they move throughout the activity.



Object flow in an activity diagram

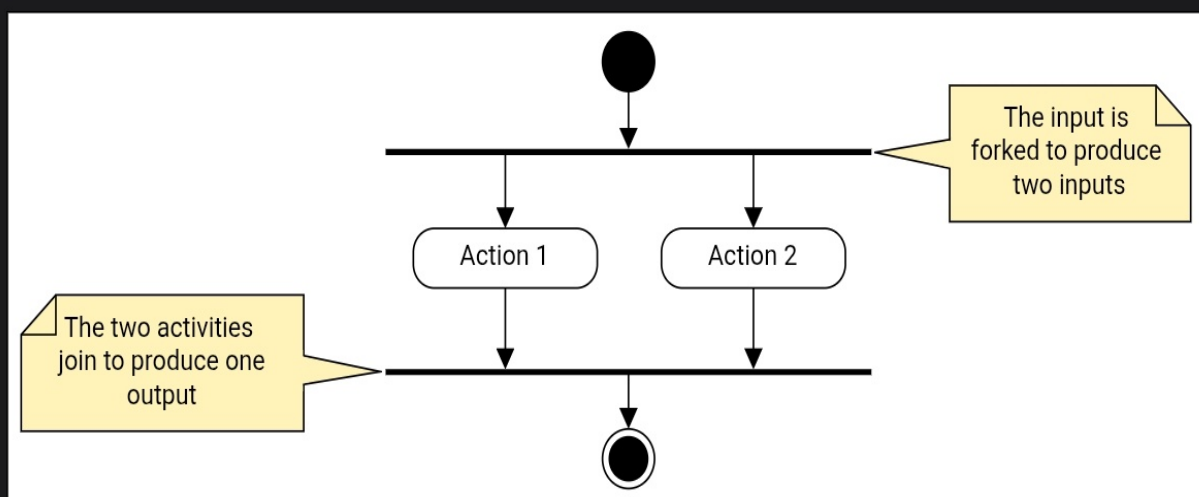
Decision: This is used to represent **multiple options that are possible** in a system. They appear as a **branch alongside the text** describing the condition for the path.

Merge: This uses the same symbol as a decision. However, this shows that multiple options join at this node, but leads to a single output.

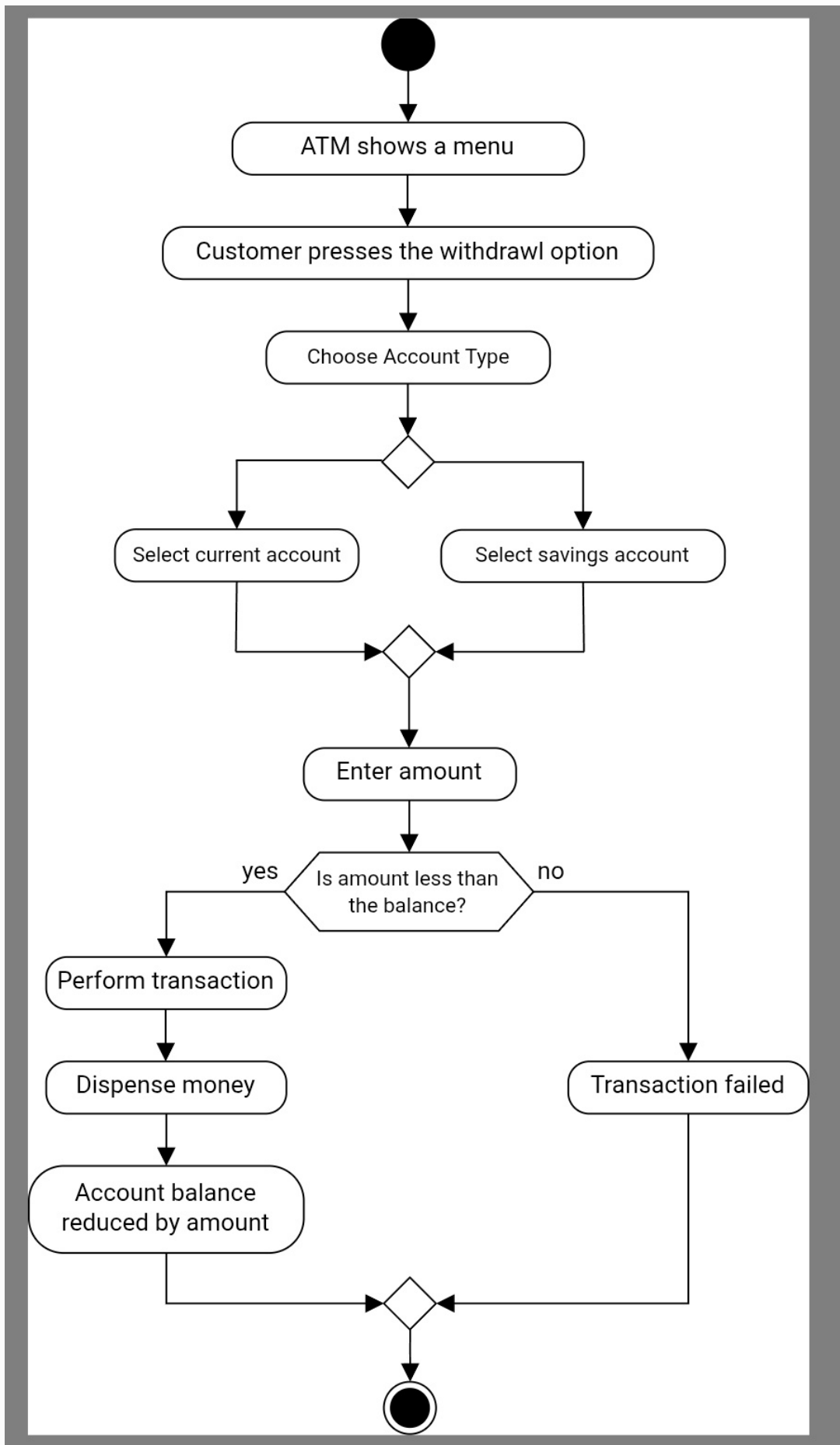


Flow final in an activity diagram

Fork and join: The **fork node** represents a **single activity that is split into two concurrent activities** happening alongside each other. On the other hand, the join node joins concurrent activities together to lead to a single activity.



Fork and join in an activity diagram



Sequence vs. activity diagram

We have now studied both the sequence and the activity diagram. As a refresher, the following table provides an overview of the differences that exist between both of them:

Sequence Diagram	Activity Diagram
It is used for dynamic modeling.	It is used for functional modeling.
It visualizes the flow of messages from one object to the other.	It visualizes the flow of messages from one activity to the other.
It helps in modeling the systems workflow.	It helps in visualizing a systems sequence of calls for a particular functionality.
It describes the behavior of various objects just for an independent use case.	It describes the general sequence of actions for various objects as well as various use cases.

Now that we have learned the conventions and benefits of these types of UML diagrams, let's test these concepts in the next lesson.